BusinessX

# Development Guide

Architecture, setup, modules, data model, security, performance, testing and deployment for the BusinessX UK platform (Laravel 12).

| | |
|---|---|
| Version | 1.0 — 13 October 2026 |
| Codebase | businessX-uk-html-mockup · branch `development` |
| Framework | Laravel 12.68 · PHP 8.2 · MySQL |
| Status | 128 automated tests passing · PHPStan level 5 clean · Pint clean |

# Contents


1. Project overview
2. Technology stack
3. Architecture
4. Repository layout
5. Local development setup
6. Configuration & environment variables
7. Functional modules
8. Payments (Razorpay)
9. Data model
10. Routes reference
11. Security model
12. Performance & caching
13. Testing & code quality
14. Coding conventions
15. Deployment checklist
16. Troubleshooting
17. Known issues & open decisions
18. Audit change log (October 2026)


> **Who this is for**
>
> Developers joining the project, reviewers, and whoever deploys or operates BusinessX. It describes the code as it stands on the `development` branch after the October 2026 security, performance and quality audit. Where something is a recommendation rather than current behaviour, it is marked as such.

# 1. Project overview

**BusinessX** (BusinessX – World Trade Council) is a UK marketplace that connects four kinds of members:

| Profile type | Purpose | Legacy code | Table |
|---|---|---|---|
| **Business** | Businesses for sale, seeking investors or loans | 1 | `profile_business` |
| **Investor** | Individuals and firms looking to invest or acquire | 2 | `profile_investor` |
| **Mentor** | Advisers offering expertise | 4 | `profile_mentors` |
| **Startup** | Early-stage companies raising funds | 7 | `profile_startups` |

Visitors browse public directory listings and profile pages. Members register one or more profiles, contact other profiles, hold conversations (with real-time live chat), exchange proposals with attachments, and can buy a paid membership (Premium or Gold) through Razorpay to gain priority placement.

### Main capabilities

**Public site**

Home page with sliders and counters, four directory listings with filters, public profile pages, articles and comments, static pages, pricing, newsletter with double opt-in, keyword chatbot.

**Accounts & registration**

Multi-step registration wizard per profile type, quick registration, email verification, first-time password setup, login, password reset.

**Member dashboard**

Account details, password, profile management, inbox and live chat, proposals sent and received, instant auto-response, activity notifications, My Plan.

**Payments**

Guest checkout from the pricing page (claimed at registration), and My Plan upgrades for existing profiles. Server-side verification plus a signed webhook.

> **Legacy schema**
> The database descends from an older PHP system. Most tables keep their legacy names and columns (for example `user_account` with primary key `user_id`, numeric profile-type codes, and tinyint status flags). Code therefore uses the query builder (`DB::table()`) far more than Eloquent models. Keep this in mind before renaming anything.

# 2. Technology stack

| Layer | Technology | Notes |
|---|---|---|
| Language | PHP 8.2+ | Local XAMPP ships PHP 8.2.12. |
| Framework | Laravel 12.68 (pinned `12.68.*`) | Slim Laravel 11+ structure: `bootstrap/app.php` configures routing, middleware and exceptions. |

| Layer | Technology | Notes |
|---|---|---|
| Database | MySQL / MariaDB (XAMPP) | Tests run on in-memory SQLite; every migration runs on both. |
| Sessions, cache, queue | Database drivers | `SESSION_DRIVER`, `CACHE_STORE`, `QUEUE_CONNECTION` = `database`. Redis recommended for cache in production. |
| Real-time | Laravel Reverb 1.6 + Laravel Echo + pusher-js | Private channel `conversation.{id}` for live chat. |
| File storage | Amazon S3 (`league/flysystem-aws-s3-v3`) | Images public (optionally via `IMG_CDN`); documents private with 5-minute signed URLs. |
| Payments | Razorpay (REST API + HMAC signatures) | Currency GBP. SDK package is installed; orders are created with Laravel's HTTP client for timeout control. |
| Mail | SMTP, queued mailables | Verification, reset, newsletter, activity notifications. |
| Frontend | Blade, Vite 7, Tailwind 4 | Most styling is hand-written CSS in `public/css`; the Vite bundle (live chat) loads on dashboard pages only. |
| Testing | PHPUnit 11 | Feature tests in `tests/Feature`. |
| Quality | Laravel Pint, Larastan 3 (PHPStan 2) level 5 | `pint.json`, `phpstan.neon`; composer scripts below. |

# 3. Architecture

## Request flow

**Browser**
Blade pages, plain JS in `public/js`, Echo client on dashboard

→

**Apache / `public/index.php`**
Root `.htaccess` blocks source & secrets

→

**Middleware**
web (session, CSRF), auth, signed, throttle

→

**Controllers**
Validate (Form Requests), authorise ownership, query

→

**Services / Support**
Payments, schema & content caches

→

**MySQL · S3 · Razorpay · SMTP · Reverb**

## Application layers

| | |
|---|---|
| Routes | `routes/web.php` (public, auth, payments, webhook), `routes/dashboard.php` (everything under `/dashboard`, all behind `auth`), `routes/channels.php` (broadcast authorisation). |
| Controllers | `app/Http/Controllers`. Listing pages share `AbstractListingController` (search, location, industry, amount ranges, membership priority, sorting, card building). `DashboardController` holds most member features. |
| Form Requests | `app/Http/Requests/Dashboard`: proposal, password and account-profile validation. Other controllers still validate inline. |
| Services | `app/Services/Payments`: `RazorpayGateway` (API + signatures) and `MembershipPaymentService` (idempotent completion, claims, membership records). |
| Support | `app/Support/SchemaCache` (cached `Schema::has*`), `app/Support/PublicContentCache` (home page and footer data). |
| Models | Mostly thin legacy models. The ones in active use: `UserAccount` (auth model), `ProfileMedia`, plus notifications via Laravel's `notifications` table. |
| Events & notifications | `ConversationMessageSent` (broadcast), `DashboardActivityNotification` (database + mail, queued after commit). |
| Views | `resources/views`: `layouts`, `pages` (public), `dashboard`, `static`, `emails`, `components`, `errors`. |

## Key design decisions

- **Query builder over Eloquent.** Legacy tables are queried with `DB::table()`. Profile types are described by arrays (table, id column, status column, title columns) in `AuthController`, `DashboardController::PROFILE_TYPES` and `MembershipPaymentService::PROFILE_TABLES`.
- **Optional legacy tables/columns.** Code checks `SchemaCache::hasTable()/hasColumn()` before using some tables, so partially migrated databases (and focused tests) keep working. Use `SchemaCache`, never the `Schema` facade, in request code: the facade queries `information_schema` on every call.
- **Authorization by ownership in queries.** Every lookup of a member-owned record filters by the signed-in user and answers **404** (not 403) when nothing matches, so responses do not reveal whether a record exists.
- **Configuration-driven registration.** `config/registration_profiles.php` defines every wizard step and field per profile type; validation rules are generated from it.

# 4. Repository layout

```
# Laravel application
app/
  Events/ConversationMessageSent.php      live-chat broadcast
  Http/Controllers/                       public, auth, dashboard, payments, webhook
  Http/Requests/Dashboard/                Form Requests
  Mail/                                   verification, reset, newsletter mailables
  Models/                                 legacy table models (UserAccount = auth model)
  Notifications/DashboardActivityNotification.php
  Providers/AppServiceProvider.php        rate limiters, password rules, view composers, caches
  Services/Payments/                      RazorpayGateway, MembershipPaymentService
  Support/                                SchemaCache, PublicContentCache
bootstrap/app.php                         routing, middleware (CSRF exception for webhook)
config/registration_profiles.php          registration wizard definition per profile type
database/migrations/                      legacy schema + new tables + indexes
database/seeders/                         demo data (see README for focused seeders)
resources/views/                          Blade templates
resources/js/app.js                       Echo / live-chat client (Vite)
routes/web.php, dashboard.php, channels.php
tests/Feature/                            feature tests (PHPUnit)
public/                                   web root: index.php, css/, js/, assets/, build/

# Static HTML mockups (design reference, not part of Laravel)
*.php in project root, includes/, css/, js/, assets/
```

> **Mockup files at the project root**
>
> The root-level `*.php` pages and `includes/` are the original static design mockups. They are still reachable on XAMPP for reference, are excluded from Pint and PHPStan, and are **not** used by the Laravel app. Do not add application logic to them.

## Supporting files

| File | Purpose |
|---|---|
| `.htaccess` (root) | Blocks web access to dotfiles, source, config, storage, vendor and tooling files when the project root is served directly (XAMPP). |
| `.env.example` | Every environment variable the app reads, without secret values. |
| `pint.json` | Laravel preset; excludes the root mockups and `includes/`. |
| `phpstan.neon` | Larastan, level 5, analyses `app/`. |
| `.git-blame-ignore-revs` | Hides the bulk formatting commit from `git blame`. |
| `phpunit.xml` | Test environment: SQLite in memory, array cache/session, sync queue, array mail. |

# 5. Local development setup

### Prerequisites

- XAMPP (Apache + MySQL/MariaDB) with PHP 8.2+, extensions: pdo_mysql, mbstring, openssl, gd (with WebP), fileinfo, curl.
- Composer 2, Node.js 20+ and npm.
- Optional: AWS S3 bucket and credentials (uploads), Razorpay test keys (payments), SMTP account (mail).

### First-time setup

```
# 1. Dependencies
composer install
npm install

# 2. Environment
copy .env.example .env          # then fill in DB, mail, AWS, Reverb, Razorpay values
php artisan key:generate

# 3. Database (create the MySQL database first)
php artisan migrate
php artisan db:seed --class=TestimonialSeeder      # optional demo data
php artisan db:seed --class=ProfileRecordsSeeder   # optional demo profiles

# 4. Frontend assets
npm run build
```

### Running the app

On XAMPP the site is served from `http://localhost/businessX-uk-html-mockup/public` (this is `APP_URL`). For the full development stack (built-in server, queue worker, log tail, Vite and Reverb together) use:

```
composer dev
# runs: php artisan serve | queue:listen | pail | npm run dev | reverb:start --port=8081
```

If you only use Apache, still run these in separate terminals when you need them:

```
php artisan queue:work            # queued mail and notifications
php artisan reverb:start --host=127.0.0.1 --port=8081   # live chat
```

> **Tip: queued jobs**
> Verification emails, password resets and activity notifications are queued. Without a worker they sit in the `jobs` table. Locally you can set `QUEUE_CONNECTION=sync` to send immediately.

# 6. Configuration & environment variables

All values live in `.env` (never committed). `.env.example` lists every key. The important ones:

| Variable(s) | Purpose | Production value |
|---|---|---|
| `APP_ENV`, `APP_DEBUG`, `APP_URL` | Environment, debug pages, base URL | `production`, `false`, https URL |
| `APP_KEY` | Encryption key (sessions, signed URLs) | Unique, never shared |
| `LOG_LEVEL` | Log verbosity | `warning` |
| `DB_*` | MySQL connection | Dedicated least-privilege user |
| `SESSION_DRIVER`, `SESSION_LIFETIME` | Database sessions, 120 minutes | `database` or `redis` |
| `SESSION_SECURE_COOKIE`, `SESSION_ENCRYPT` | HTTPS-only cookie, encrypted payload | `true`, `true` |
| `CACHE_STORE` | Cache backend | `redis` recommended |
| `QUEUE_CONNECTION` | Queue backend | `database` or `redis` + supervised worker |
| `MAIL_*` | SMTP delivery and sender | Real SMTP provider |
| `AWS_ACCESS_KEY_ID`, `AWS_SECRET_ACCESS_KEY`, `AWS_DEFAULT_REGION`, `AWS_BUCKET`, `AWS_URL`, `AWS_ENDPOINT` | S3 storage for profile media and proposal attachments | IAM user limited to the bucket |
| `IMG_CDN` | Optional CDN base URL for public images | CDN URL or empty |
| `BROADCAST_CONNECTION`, `REVERB_*`, `VITE_REVERB_*` | Reverb server and the client settings baked into the Vite build | `reverb`, wss on 443 |
| `RAZORPAY_KEY_ID`, `RAZORPAY_KEY_SECRET` | Razorpay API credentials. Payments are disabled (friendly message) while blank. | Live keys |
| `RAZORPAY_WEBHOOK_SECRET` | Secret configured on the Razorpay webhook. The webhook answers 503 while blank. | Required |

> **After changing .env in production**
> Run `php artisan config:cache` again, and rebuild assets (`npm run build`) if any `VITE_*` value changed.

## Configuration files worth knowing

- `config/registration_profiles.php` – wizard steps and fields per profile type (labels, types, options, required flags, upload `accept` lists). Validation rules are derived from it in `AuthController`.
- `config/services.php` – `razorpay.key`, `razorpay.secret`, `razorpay.webhook_secret`.
- `config/filesystems.php` – `s3` disk and `cdn_url`.
- `app/Providers/AppServiceProvider.php` – the `registration` rate limiter (10/min per IP), `Password::defaults()`, view composers, cache wiring.

# 7. Functional modules

## 7.1 Public site

| Feature | Controller | Behaviour |
|---|---|---|
| Home page | `HomeController` | Industry and city counters, investment ranges, 4 latest articles, business/investor/startup/mentor sliders (12 each), 3 testimonials. Whole dataset cached (see section 12). |
| Directory listings | `Business/Investor/Startup/MentorListingController` → `AbstractListingController` | Keyword, location, industry, intent and amount filters; paid members first; 12 per page with query-preserving pagination. |
| Profile pages | `ProfileContactController@show` | Public detail page; private contact fields hidden; each view by a signed-in member is recorded in `profile_visitors`. |
| Articles | `ArticleController`, `ArticleCommentController` | Search, tag and date filters, 9 per page; detail page sanitises stored HTML with an allow-list; newest 50 approved comments; members may comment (moderated). |
| Static pages | `StaticPageController` | About, Disclaimer, Privacy, Terms, Contact (content defined in the controller). |
| Newsletter | `SubscribeController`, `NewsletterVerificationController` | Double opt-in: status `P` until the signed link (24 h) is followed, then `S` plus a confirmation email. |
| Contact form | `ContactController` | Stores to `businessex_contactus`. |
| Chatbot | `ChatbotController` | Keyword scoring against `chatbot_knowledge_base` (cached 5 min); lead capture to `chatbot_leads`. |
| City suggestions | `CitySuggestionController` | Autocomplete from `bx_cities` (UK = country 5). |

## 7.2 Registration & authentication

The auth model is `App\Models\UserAccount` (table `user_account`, key `user_id`) on the standard `web` session guard. Passwords use Laravel's `hashed` cast (bcrypt).

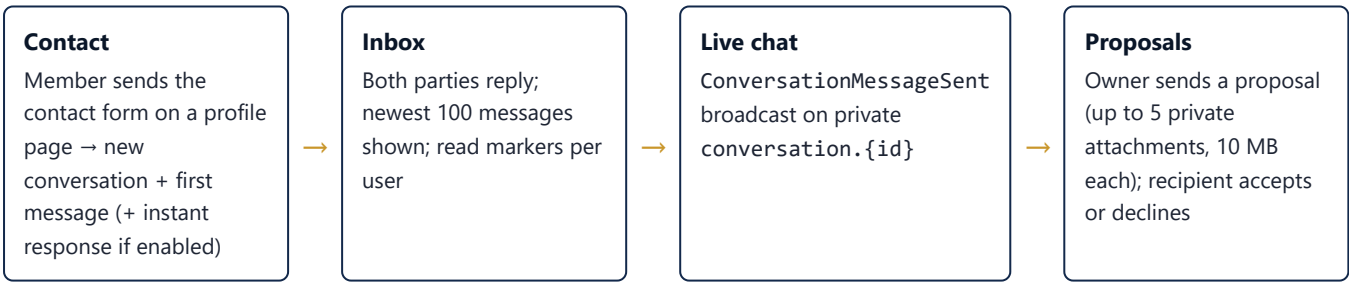
1. **Register.** Full wizard (`/register/{type}`) or homepage quick registration (`/register/quick`). Creates `user_account` (no password yet), the profile row, `user_profiles` mapping and stored uploads in a transaction. A signed-in member can add further profiles from the dashboard without a new account.
2. **Verify email.** A queued email carries a signed link (60 min). Verifying redirects a password-less account to the set-password page.
3. **Set password.** Signed link, valid only while the account still has no password (single use). Inactive accounts are refused.
4. **Login.** Throttled 5 attempts per email+IP; requires a verified email and an active account; session regenerated.
5. **Password reset.** Laravel password broker (hashed, single-use, 60-minute tokens). A successful reset ends every stored session of the account.

> **Validation rules to know**
> Full registration requires UK mobiles in `07XXXXXXXXX` format, rejects `example.com`, `sample.com` and `test.com` addresses, requires city fields to match `bx_cities`, and validates UK postcodes. Uploads: max 1 MB, extension allow-list from the config, image dimension limits. Passwords (reset/setup): at least 8 characters with letters and numbers; the dashboard change form also requires a symbol.

## 7.3 Profiles & media

- Uploads are stored on S3 under `profiles/{type}/images|documents/Y/m/{uuid}.{ext}`. The extension comes from the file content, never the client name.
- Images are public (served from S3 or `IMG_CDN`). Documents are private and only served through `/profile-media/{id}/download` (owner only) as a 5-minute signed S3 URL.
- Every upload is recorded in `profile_media`; the profile row keeps the current URL in its legacy column (for example `seller_prof_pic`).
- Owners edit profiles at `/dashboard/profiles/{type}/{id}/edit`; ownership is enforced in the query.

## 7.4 Messaging, live chat & proposals




- Tables: `profile_contact_conversations`, `profile_contact_messages`, `profile_contact_conversation_reads`, `profile_proposals`, `profile_proposal_attachments`, `profile_instant_responses`.
- Channel authorisation (`routes/channels.php`) admits only the two participants of a conversation.
- Accept/decline uses a conditional update in a transaction, so only one response wins and notifies. An identical proposal resubmitted within a minute is treated as a double submit.
- `DashboardActivityNotification` (database + mail) is queued **after commit** and feeds the header notification menu.

### 7.5 Member dashboard

| Page | What it does |
|---|---|
| `/dashboard` | Account summary, owned profiles with view counts, quick actions. |
| `/dashboard/profile` | Edit name, location, designation, company. Email and mobile are locked. |
| `/dashboard/password` | Change password; signs the user out everywhere afterwards. |
| `/dashboard/profiles/...` | Create, view and edit owned profiles. |
| `/dashboard/inbox`, `/live-chat/...` | Conversations, replies, live-chat JSON endpoints. |
| `/dashboard/proposals/sent\|received` | Proposal lists with status filter (15 per page), composer, attachment downloads. |
| `/dashboard/instant-response` | Automatic first reply to new contacts. |
| `/dashboard/my-plan` | Membership details and upgrade checkout (Premium / Gold; Free and Platinum are informational). |
| `/dashboard/activity` | Notification history; mark one or all as read. |

## 8. Payments (Razorpay)

Prices are fixed on the server in `MembershipPaymentService::PLAN_AMOUNTS` (pence, GBP). The client never supplies an amount.

| Plan | Monthly | Annual | Profile `membership_plan` code |
|---|---|---|---|
| Premium | £29.00 | £299.00 | 1 |
| Gold | £89.00 | £999.00 | 2 |

### Flow

**1. Initiate**
POST `.../payment/initiate` creates a Razorpay order (15 s timeout) and an `online_payments` row, status 0

→

**2. Checkout**
Razorpay Checkout in the browser (card / UPI)

→

**3. Verify**
POST `.../payment/verify` checks the HMAC signature, then `complete()`

→

**4. Webhook**
POST `/webhooks/razorpay` (`payment.captured`, `order.paid`) as the safety net

### Two entry points

- **Guest checkout** (`/pricing`): the payment is created with `profile_id = 0`. After verification the browser is sent to the registration wizard with `payment_order_no`. Registration claims the payment for the new profile only if the order was verified in the same session **or** the account email equals the payer email. The claim is atomic (row lock + `profile_id = 0` guard).
- **My Plan upgrade** (`/dashboard/my-plan`): ownership of the profile is checked first; on completion the profile gets `membership_paid = 1` and its plan code, the previous active `profile_memberships` row is deactivated and a new one (1 month or 1 year) is inserted.

### Guarantees

| Risk | How it is handled |
|---|---|
| Replay of a valid signature | `complete()` locks the payment row and returns early if it is already SUCCESS; `razorpay_payment_id` is unique. |
| Webhook and browser both arrive | Same idempotent `complete()`; whichever is second is a no-op. |
| Browser closed after capture | Webhook activates the plan. |
| Forged or wrong amount in webhook | HMAC check with `RAZORPAY_WEBHOOK_SECRET`; amount and currency must match the stored order. |
| Bad signature on a paid order | Only INITIATED payments can be marked FAILED. |
| Partial writes | Status, profile and membership updates run in one transaction. |

Statuses in `online_payments.payment_status`: 0 initiated, 1 success, 2 failed. `udf` holds the Razorpay order id. The webhook route is exempt from CSRF in `bootstrap/app.php` because Razorpay cannot send a token; its HMAC signature replaces it.

> **Razorpay dashboard setup**
> Create a webhook pointing to `https://<your-domain>/webhooks/razorpay` with events `payment.captured` and `order.paid`, and put its secret in `RAZORPAY_WEBHOOK_SECRET`.

## 9. Data model

About 90 tables. The ones the application actively uses, grouped by area:

| Area | Tables | Notes |
|---|---|---|
| Accounts | `user_account`, `user_profiles`, `password_reset_tokens`, `sessions` | `user_profiles` maps users to profiles (legacy type code + profile key). |
| Profiles | `profile_business`, `profile_business_mgmt`, `profile_investor`, `profile_mentors`, `profile_startups`, `profile_media`, `profile_visitors` | Status columns: `business_profile_status`, `inv_profile_status`, `mentor_profile_status`, `startup_profile_status` (1 = active). |
| Reference data | `industry_categories`, `ind_pref_business`, `bx_cities`, `country` | Category tree via `parent_id`; UK cities have `country = 5`. |
| Messaging | `profile_contact_conversations`, `profile_contact_messages`, `profile_contact_conversation_reads`, `profile_instant_responses` | Read markers keyed by (conversation, user). |
| Proposals | `profile_proposals`, `profile_proposal_attachments` | Status: pending / accepted / declined. |
| Payments | `online_payments`, `profile_memberships`, `membership_plans`, `bx_coupon` | Unique `order_no` and `razorpay_payment_id`. Coupons exist in the schema but are not implemented. |

| Area | Tables | Notes |
|---|---|---|
| Content | `bx_articles`, `bx_author`, `articles_comments`, `testimonials` | Maintained outside this app (admin), except comments. |
| Marketing | `businessex_newsletter`, `businessex_contactus`, `chatbot_knowledge_base`, `chatbot_leads` | Newsletter status: P pending, S subscribed, U unsubscribed. |
| Framework | `notifications`, `jobs`, `failed_jobs`, `job_batches`, `cache`, `cache_locks`, `migrations` | Standard Laravel. |

### Indexes added for the workload

- Listings and search: status + created_at + id per profile table, `user_id`, startup industry, category parent, article status orders.
- Messaging: conversation owner/sender + updated_at, messages by conversation, proposals by sender/recipient + status + created_at.
- Payments: unique `order_no`, unique `razorpay_payment_id`, index on `udf`, membership owner index.
- Comments (`article_id, comment_status, created_at`) and profile visitors (`profile_type, profile_id`).

### Migration rules

- Never edit a migration that has run anywhere; add a new one. Every migration must run on MySQL **and** SQLite (tests run all of them).
- Provide a working `down()`. Check data before adding unique keys (see `2026_10_13_000001` for the pattern).
- Running migrations automatically clears the schema cache (`SchemaCache` listens to `MigrationsEnded`).
- There are no foreign keys in the legacy schema; integrity is enforced in code. Add constraints only after cleaning orphaned data.

## 10. Routes reference

Run `php artisan route:list --except-vendor` for the live list (90 routes). Legacy `.php`-style URLs redirect to their new routes.

### Public

| Route | Purpose | Middleware |
|---|---|---|
| `GET /` | Home | – |
| `GET /{business\|investor\|startup\|mentor}-listing` | Directory listings | – |
| `GET /profile-details/{type}/{id}` | Public profile page | – |
| `GET\|POST /profile-details/{type}/{id}/contact` | Contact a profile (starts a conversation) | auth, throttle 5/min |
| `GET /article`, `GET /article/{article}` | Articles | – |
| `POST /article/{article}/comments` | Comment | auth, throttle 5/min |

| Route | Purpose | Middleware |
|---|---|---|
| `GET /about-us`, `/disclaimer`, `/privacy-policy`, `/terms-and-conditions`, `/contact-us`, `/pricing` | Static pages | – |
| `POST /newsletter/subscribe`, `GET /newsletter/verify/{id}/{hash}` | Newsletter | throttle / signed |
| `POST /chatbot/message`, `POST /chatbot/leads` | Chatbot | throttle 20 / 5 per min |
| `GET /uk-cities/suggest` | City autocomplete | throttle 60/min |

### Accounts

| Route | Purpose | Middleware |
|---|---|---|
| `GET /{type}-registration`, `POST /register/{type}`, `POST /register/quick` | Registration | throttle:registration |
| `GET\|POST /login`, `POST /logout` | Session | throttle 5/min; auth |
| `GET /email/verify/{id}/{hash}`, `POST /email/verification-notification` | Email verification | signed; throttle 3/hour |
| `GET\|POST /account/set-password/{id}/{hash}` | First-time password | signed, throttle |
| `GET\|POST /forgot-password`, `GET /reset-password/{token}`, `POST /reset-password` | Password reset | throttle 5/min |
| `GET /profile-media/{media}/download` | Private document download | auth (owner) |

### Payments

| Route | Purpose | Middleware |
|---|---|---|
| `POST /pricing/payment/initiate\|verify` | Guest checkout | throttle 10/min |
| `POST /dashboard/my-plan/payment/initiate\|verify` | Upgrade checkout | auth, throttle 10/min |
| `POST /webhooks/razorpay` | Razorpay webhook (no CSRF; HMAC) | throttle 120/min |

### Dashboard (all behind `auth`)

| Route | Purpose |
|---|---|
| `GET /dashboard`, `/activity`, `POST /notifications/{id}/read`, `/notifications/read-all` | Home, notifications |
| `GET\|PUT /dashboard/profile`, `GET\|PUT /dashboard/password` | Account details, password |
| `GET /dashboard/profiles/create[/{type}]`, `POST /dashboard/profiles/create/{type}` | Add a profile |

| Route | Purpose |
|---|---|
| `GET /dashboard/profiles/{type}/{id}[/edit]`, `PUT /dashboard/profiles/{type}/{id}` | View / edit owned profile |
| `GET /dashboard/inbox[/{conversation}]`, `POST .../reply` (30/min), `POST .../proposals` (10/min) | Inbox, replies, proposals |
| `GET /dashboard/live-chat/conversations[/{id}/messages]` | Live-chat JSON |
| `GET /dashboard/proposals/sent\|received`, `POST /dashboard/proposals`, `PUT .../{proposal}/status`, `GET .../attachments/{attachment}` | Proposals |
| `GET\|POST /dashboard/instant-response` | Auto-reply settings |
| `GET /dashboard/my-plan` | Membership |

# 11. Security model

| Concern | Implementation |
|---|---|
| Web root exposure | Production must serve `public/` only. On XAMPP the root `.htaccess` denies `.env`, `.git`, `storage`, `vendor`, `app`, `config` and tooling files (verified 403). |
| Authentication | Session guard, bcrypt via the `hashed` cast, login throttling, session regeneration on login, invalidation on logout, verified + active account required. |
| Account recovery | Broker tokens (hashed, single-use, 60 min); single-use set-password link; reset and password change end all sessions; no user enumeration in messages. |
| Authorization / IDOR | Every member-owned lookup is filtered by the signed-in user and returns 404 otherwise; broadcast channel limited to conversation participants; media downloads owner-only. |
| CSRF | Enabled for all web routes; the only exception is the HMAC-verified Razorpay webhook. |
| Validation & mass assignment | Validated arrays only; no `$request->all()`; status/amount/activation fields are not fillable on models. |
| SQL injection | Query builder bindings everywhere; raw fragments use hard-coded column names; sort and filter values are allow-listed. |
| XSS | Blade escaping; the only unescaped output is article HTML passed through a DOM allow-list sanitiser and hard-coded static-page content. |
| Uploads | Size limits, extension allow-lists, image dimension limits, UUID names, extension from content, private documents via short-lived signed URLs. |
| Rate limiting | Login, reset, verification, registration, contact, comments, replies, proposals, payments, chatbot, newsletter, city suggestions. |
| Payments | Server-side amounts, HMAC signature checks with `hash_equals`, idempotent completion, webhook amount/currency check. |
| Secrets | Only in `.env` (untracked); no secrets in logs or responses; Razorpay errors logged by message only. |

**Rules for new code**

- Never disable CSRF, validation or ownership checks to make something work.
- Scope every query for member data by `Auth::id()` and answer 404 when nothing matches.
- Add a `throttle` middleware to any new endpoint that sends email, writes data publicly or calls an external API.
- Store uploads on S3 with generated names; never trust the client filename or MIME type.

# 12. Performance & caching

## Caches

| Cache | Key / TTL | Invalidation |
|---|---|---|
| `SchemaCache` (table/column metadata) | `schema-cache:v1`, 24 h; memoised per request | Automatic on every `migrate` / rollback; or `php artisan cache:clear`. |
| Home page data | `public:home:v1`, 5 min | Cleared by every profile write in the app (registration, quick registration, dashboard profile edit, payment claim/activation). Articles and testimonials rely on the TTL. |
| Footer / listing reference data (industry tree, UK cities) | `public:layout:v1`, 1 h | TTL, or `cache:clear` after editing categories or cities. |
| Chatbot knowledge base | `chatbot.active-knowledge-base.v1`, 5 min | TTL. |
| Listing industry categories | in `AbstractListingController` | TTL. |

## Measured results (dev database, warm cache, queries per request)

| Page | Before audit | After |
|---|---|---|
| Home | 40 | 0 (+1 cache read) |
| Directory listing / search | 9–10 / 16 | 2 / 1 |
| Static pages, login, pricing, registration forms | 2 | 0 |
| Article index / detail | 9 / 14 | 4 / 7 |
| Dashboard / My Plan | 31 / 20 | 15 / 9 |
| Inbox and proposal lists | grew with rows (52–82 at 24 rows) | constant |

## Performance rules for new code

- No queries inside loops: batch with `whereIn()` + `keyBy()` (see `profileConversationLabels()`). `DashboardQueryCountTest` fails if list pages start growing with their rows.
- Paginate or cap every list (listings 12, articles 9, proposals 15, comments 50, thread messages 100).
- Use `SchemaCache`, not `Schema`, in request code.

- Select the columns you need on large tables; check new filters with `EXPLAIN` before adding indexes.
- Call external APIs with explicit timeouts, and queue slow work (mail is already queued).
- If you add public, visitor-independent data to the home page, it is cached; clear `PublicContentCache` wherever that data is written.

### Frontend

- Vite bundles only `resources/js/app.js` (live chat, about 81 KB / 23 KB gzip) and `resources/css/app.css`; loaded on dashboard pages.
- Page styles and scripts are plain files in `public/css` and `public/js`, cache-busted with `?v=` query strings.
- Keep images at display size (default profile images are 600 px wide).

## 13. Testing & code quality

### Commands

```
composer test                 # or: php artisan test          (128 tests)
php artisan test --filter=MembershipPaymentTest
composer analyse              # PHPStan / Larastan level 5
composer format               # apply Laravel Pint
composer format:check         # CI-style formatting check
php artisan route:list --except-vendor
```

> **Definition of done for a change**
>
> `composer test`, `composer analyse` and `composer format:check` all pass, new behaviour has a feature test (including the unauthorised / invalid case), and migrations run up and down.

### How the tests are built

- Environment from `phpunit.xml`: SQLite in memory, array cache and session, sync queue, array mailer, broadcasting off.
- Many older tests create only the tables they need with `Schema::create()` in `setUp()`. Newer tests run the real migrations (`$this->artisan('migrate')`), which is preferred because hand-written schemas drift from production.
- Use `Mail::fake()`, `Http::fake()` (Razorpay API) and `Storage::fake('s3')`; never call real services.
- Payment tests sign callbacks with a test secret through `hash_hmac('sha256', ...)`.

### Notable test files

| File | Covers |
|---|---|
| `AuthenticationTest`, `AccountSecurityTest` | Registration rules, verification, login, reset, set-password link, session termination, inactive accounts |
| `MembershipPaymentTest` | Replay protection, ownership, signature failures, guest claims, webhook, server pricing |
| `ProfileContactFlowTest` | Contact, inbox, proposals, races, double submits, attachment downloads |

| File | Covers |
| --- | --- |
| `DashboardRoutesTest`, `DashboardQueryCountTest` | Dashboard pages and N+1 regression guard |
| `SchemaCacheTest`, `PublicContentCacheTest` | Cache behaviour and invalidation paths |
| `Home*Test`, `ListingControllersTest`, `ArticlePagesTest`, `NewsletterSubscriptionTest` | Public pages |

# 14. Coding conventions

- **Formatting:** Laravel Pint (PSR-12 based Laravel preset). Run `composer format` before committing.
- **Types:** parameter and return types on new methods; generics in docblocks for collections (`Collection<int, \stdClass>`).
- **Controllers stay thin:** HTTP concerns only. Put multi-step business logic in `app/Services` (see payments) and validation in Form Requests (`app/Http/Requests`).
- **Transactions:** wrap multi-table writes in `DB::transaction()`; use conditional updates or `lockForUpdate()` for anything that can race; use `upsert()` instead of exists-then-insert.
- **Notifications and mail** are queued; notifications sent inside transactions must use `afterCommit()` (already set on `DashboardActivityNotification`).
- **Profile type handling:** reuse the existing type maps; do not hard-code table names in new places.
- **Comments** explain *why* (constraints, legacy quirks), not what the next line does.
- **Commits:** small, one concern each, imperative subject line, body explaining the reason and any behaviour change. Formatting-only changes go in their own commit.
- **Branches:** work on feature branches off `development`; merge to `main` through pull requests.

# 15. Deployment checklist

### Server

- Web server document root points to `public/` (do not rely on the root `.htaccess` in production).
- HTTPS everywhere; `APP_DEBUG=false`; `SESSION_SECURE_COOKIE=true`; `LOG_LEVEL=warning`.
- `storage/` and `bootstrap/cache/` writable by the web user; nothing else writable.
- Supervised queue worker (`php artisan queue:work --tries=3`) and Reverb (`php artisan reverb:start`) behind TLS.
- Scheduler cron if scheduled tasks are added: `* * * * * php artisan schedule:run`.

### Each release

```
git pull
composer install --no-dev --optimize-autoloader
npm ci && npm run build
php artisan migrate --force
php artisan config:cache
php artisan route:cache
php artisan view:cache
php artisan cache:clear       # refresh home/footer/schema caches
php artisan queue:restart
```

### First production release after the October 2026 audit

1. Rotate every secret in `.env` (DB, SMTP, AWS, Reverb, Razorpay) if the project root was ever web-accessible.
2. Set `RAZORPAY_KEY_ID`, `RAZORPAY_KEY_SECRET`, `RAZORPAY_WEBHOOK_SECRET` and create the webhook in Razorpay.
3. Run the pending migrations: `2026_10_10_072734` (document/image columns), `2026_10_13_000001` (payment keys, membership history) and `2026_10_13_000002` (comment and visitor indexes). The first of the new ones stops with a clear message if duplicate order numbers exist.
4. Prefer Redis for cache (and optionally sessions/queue) in production.

## 16. Troubleshooting

| Symptom | Likely cause and fix |
|---|---|
| Verification / reset emails never arrive | No queue worker running. Start `php artisan queue:work` or set `QUEUE_CONNECTION=sync` locally. Check `failed_jobs`. |
| "Online payment is not available yet" | Razorpay keys missing from `.env` (then `config:clear`). |
| Webhook returns 503 / 400 | 503: `RAZORPAY_WEBHOOK_SECRET` not set. 400: secret differs from the one configured in Razorpay. |
| Paid but plan not active | Check `online_payments` status and webhook delivery logs in Razorpay; a guest payment is attached only when the payer registers. |
| Live chat says "Connecting..." forever | Reverb not running, or `VITE_REVERB_*` differ from the server; rebuild assets after changing them. |
| Uploads fail with storage error | AWS credentials or bucket missing; run `config:clear` after fixing. |
| New column or table "not found" by the app after a manual DB change | Schema metadata is cached; run `php artisan cache:clear` (migrations clear it automatically). |
| Home page shows old profiles/articles | Articles/testimonials refresh within 5 minutes; run `cache:clear` to publish immediately. |
| Registration rejects a valid-looking city | Cities must exist in `bx_cities`; add the city there. |
| Tests fail with "no such table/column" | The test's hand-written schema lacks it; add it, or switch the test to `$this->artisan('migrate')`. |

## 17. Known issues & open decisions

| Prio | Item | Detail / suggestion |
|---|---|---|
| Decide | Quick vs full registration rules | Quick registration accepts `+44` numbers and `example.com` emails; full registration rejects both. Align them. |
| Decide | Guest payments in My Plan | A claimed guest payment marks the profile as paid but does not create a `profile_memberships` row, so "Membership Details" stays empty and no expiry is tracked. |
| Decide | Home page counts | 4 articles and 3 testimonials are shown; earlier documentation said 12 and 6. Confirm the intended numbers. |
| Decide | Replaced media files | Replacing a profile image or document keeps the old S3 object and `profile_media` row forever. Add cleanup after a successful replace. |
| Decide | Profile view counting | Every view inserts a row; refreshing inflates counts. Count one view per viewer per profile per day. |
| Decide | Contact form identity | Sender name/email are free text and each submit starts a new conversation and email. Prefill from the account and reuse open conversations. |
| Improve | `DashboardController` size | About 1,700 lines. Continue splitting into focused controllers (Inbox, Proposals, Profiles, Plan) with Form Requests. |
| Improve | Article lists load full bodies | Needed for reading time; store a word count column and select only listing columns. |
| Improve | Search | Leading-wildcard `LIKE` across many columns cannot use indexes; consider FULLTEXT indexes or Laravel Scout as data grows. |
| Improve | Foreign keys | None in the legacy schema; add after cleaning orphans, starting with payments and memberships. |
| Improve | Static analysis level | Level 5 is clean; level 6 reports about 200 missing type annotations to work through gradually. |
| Improve | Coupons | Schema exists but no logic. When implemented, increment redemptions atomically inside the payment transaction. |
| Improve | CI | Add a pipeline running `composer test`, `composer analyse` and `composer format:check` on every pull request. |

## 18. Audit change log (October 2026)

Commits on `development` after `3355d37`, oldest first.

| Commit | Phase | Change |
|---|---|---|
| `ef199bf` | Security | Block web access to secrets and source at the project root; add `.env.example`. |
| `241e8f2` | Payments | Idempotent Razorpay completion, signed webhook, atomic guest claims, HTTP timeouts, payment keys migration, throttles. |

| Commit | Phase | Change |
|---|---|---|
| `4a8c9cf` | Accounts | Single-use set-password link, sessions ended on reset, shared password rules, inactive-account checks. |
| `327d7b9` | Tests | Test suite brought back to green (20 outdated tests aligned with the application). |
| `579942b` | Performance | Schema introspection cache; dashboard N+1 queries removed with a regression test. |
| `30b1137` | Performance | Home page and footer reference data cached with invalidation on profile writes. |
| `c7fb9ca` | Performance | Comment and profile-visitor indexes (verified with `EXPLAIN`); capped comments and threads. |
| `a8d93f4` | Frontend | JS bundle 133 → 81 KB, duplicate WebSocket connection removed, placeholder images 1.3 MB → 360 KB. |
| `acc5bb9` | Performance | Skip batched lookups on empty lists. |
| `f687eac` | Integrity | Proposal races, double submits, atomic upserts, non-ASCII attachment downloads, notifications after commit, model fixes. |
| `9c652ef`, `d326111` | Quality | Laravel Pint formatting (no behaviour change) and blame-ignore file. |
| `ff54d66` | Quality | Larastan level 5 with all findings fixed (no baseline); composer scripts. |
| `fd31aea` | Quality | Dashboard validation moved into Form Requests. |

> **Keeping this document current**
>
> Update this guide in the same pull request as any change to setup steps, environment variables, routes, payment flow, caching or deployment. Outdated documentation is treated as a bug.